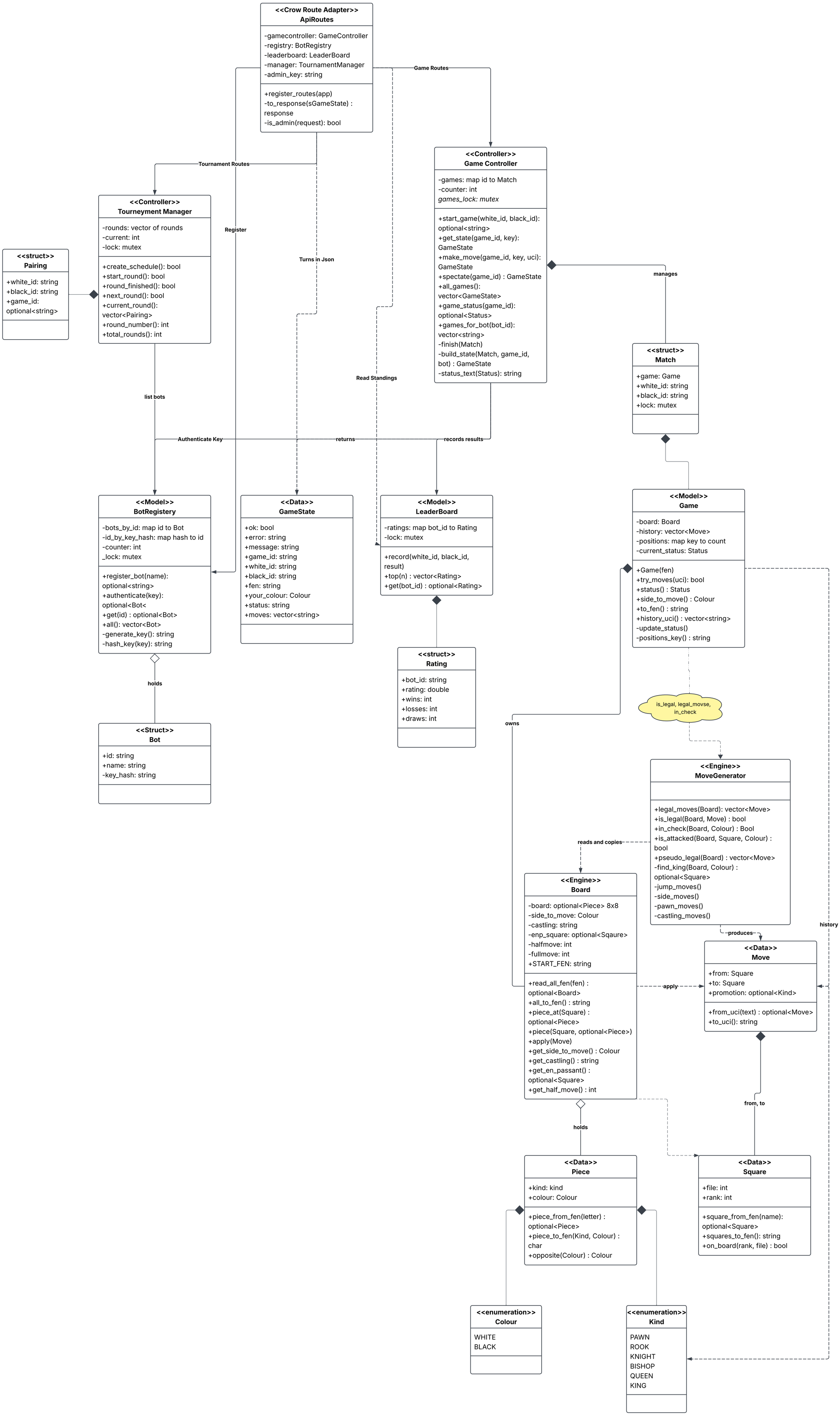

<<Crow Route Adapter>>
ApiRoutes
-gamecontroller: GameController
-registry: BotRegistry
-leaderboard: LeaderBoard
-manager: TournamentManager
-admin_key: string
+register_routes(app)
-to_response(sGameState) : response
-is_admin(request): bool
Game Routes
Tournament Routes
Register
Turns in Json
Read Standings
<<Controller>>
Game Controller
-games: map id to Match
-counter: int
games_lock: mutex
+start_game(white_id, black_id): optional<string>
+get_state(game_id, key): GameState
+make_move(game_id, key, uci): GameState
+spectate(game_id) : GameState
+all_games(): vector<GameState>
+game_status(game_id): optional<Status>
+games_for_bot(bot_id): vector<string>
-finish(Match)
-build_state(Match, game_id, bot) : GameState
-status_text(Status): string
manages
<<Controller>>
Tourneyment Manager
-rounds: vector of rounds
-current: int
-lock: mutex
+create_schedule(): bool
+start_round(): bool
+round_finished(): bool
+next_round(): bool
+current_round(): vector<Pairing>
+round_number(): int
+total_rounds(): int
<<struct>>
Pairing
+white_id: string
+black_id: string
+game_id: optional<string>
<<struct>>
Match
+game: Game
+white_id: string
+black_id: string
+lock: mutex
list bots
Authenticate Key
returns
records results
<<Model>>
BotRegistery
-bots_by_id: map id to Bot
-id_by_key_hash: map hash to id
-counter: int
_lock: mutex
+register_bot(name): optional<string>
+authenticate(key): optional<Bot<
+get(id) : optional<Bot>
+all(): vector<Bot>
-generate_key(): string
-hash_key(key): string
<<Data>>
GameState
+ok: bool
+error: string
+message: string
+game_id: string
+white_id: string
+black_id: string
+fen: string
+your_colour: Colour
+status: string
+moves: vector<string>
<<Model>>
LeaderBoard
-ratings: map bot_id to Rating
-lock: mutex
+record(white_id, black_id, result)
+top(n) : vector<Rating>
+get(bot_id) : optional<Rating>
<<Model>>
Game
-board: Board
-history: vector<Move>
-positions: map key to count
-current_status: Status
+Game(fen)
+try_moves(uci): bool
+status() : Status
+side_to_move() : Colour
+to_fen() : string
+history_uci() : vector<string>
-update_status()
-positions_key() : string
holds
<<struct>>
Rating
+bot_id: string
+rating: double
+wins: int
+losses: int
+draws: int
is_legal, legal_movse, in_check
owns
<<Struct>>
Bot
+id: string
+name: string
-key_hash: string
<<Engine>>
MoveGenerator
+legal_moves(Board): vector<Move>
+is_legal(Board, Move) : bool
+in_check(Board, Colour) : Bool
+is_attacked(Board, Square, Colour) : bool
+pseudo_legal(Board) : vector<Move>
-find_king(Board, Colour) : optional<Square>
-jump_moves()
-side_moves()
-pawn_moves()
-castling_moves()
reads and copies
produces
history
<<Engine>>
Board
-board: optional<Piece> 8x8
-side_to_move: Colour
-castling: string
-enp_square: optional<Sqaure>
-halfmove: int
-fullmove: int
+START_FEN: string
+read_all_fen(fen) : optional<Board>
+all_to_fen() : string
+piece_at(Square) : optional<Piece>
+piece(Square, optional<Piece>)
+apply(Move)
+get_side_to_move() : Colour
+get_castling() : string
+get_en_passant() : optional<Square>
+get_half_move() : int
apply
<<Data>>
Move
+from: Square
+to: Square
+promotion: optional<Kind>
+from_uci(text) : optional<Move>
+to_uci(): string
from, to
holds
<<Data>>
Piece
+kind: kind
+colour: Colour
+piece_from_fen(letter) : optional<Piece>
+piece_to_fen(Kind, Colour) : char
+opposite(Colour) : Colour
<<Data>>
Square
+file: int
+rank: int
+square_from_fen(name): optional<Square>
+squares_to_fen(): string
+on_board(rank, file) : bool
<<enumeration>>
Colour
WHITE
BLACK
<<enumeration>>
Kind
PAWN
ROOK
KNIGHT
BISHOP
QUEEN
KING